

CodeHire

INTERVIEW REPORT CARD

CANDIDATE

Dhrumil Kayastha

INTERVIEWER

codehire

DATE

March 19, 2026

PROBLEM

Two Sum

DURATION

0 min

SESSION ID

cmmxsaz3w0001u2ooppm3tggh

Performance Score

2/2

AUTO SCORE



HOST RATING (4/5)

9/10

AI RATING

PENDING

Final Code Submitted

```
// No code submitted for this problem
```

AI Code Review

The candidate provided a correct and efficient solution to the Two Sum problem within the allotted time. The code is clean, well-commented, and demonstrates a good understanding of common data structures for this problem.

TIME COMPLEXITY

SPACE COMPLEXITY

O(n) with brief explanation: The solution uses a hash map (JavaScript object) to store seen numbers and their indices. Each number is iterated through once, and hash map lookups/insertions are $O(1)$ on average.

O(n) with brief explanation: In the worst-case scenario, all numbers in the input array might be stored in the hash map if no pair is found until the very end of the array.

CODE QUALITY

Excellent

PROBLEM SOLVING APPROACH

The candidate employed a hash map (JavaScript object) to store each number encountered along with its index. For each number, they calculated the required 'complement' (target - current number) and checked if this complement already existed in the hash map. If found, the indices of the current number and the complement were returned.

STRENGTHS

- ✓ **Correctness:** The code correctly identifies the pair of numbers that sum to the target.
- ✓ **Efficiency:** The chosen approach with a hash map is optimal for this problem in terms of time complexity.
- ✓ **Readability:** The code is well-formatted and includes helpful comments, making it easy to understand.

IMPROVEMENTS

- 💡 **Error Handling (Minor):** While not strictly required for this problem in typical interview settings, adding a check for an empty input array or no solution found could be considered for more robust code.
- 💡 **Variable Naming (Minor):** While `map` and `num` are acceptable, more descriptive names like `complementMap` and `currentNumber` could slightly enhance clarity for very complex scenarios, though it's not a significant issue here.

OVERALL AI RATING

Recommendation: Strong Hire

AI Hints Provided (5)

Think about how you can efficiently check if the "complement" of a number (target - current_number) has already been seen. Consider using a data structure that allows for quick lookups.

Think about how you can efficiently check if a complement number exists for each element you encounter. Consider using a data structure that allows for quick lookups.

Think about how you can efficiently check if the complement of the current number exists within the array. Consider using a data structure that allows for quick lookups.

Think about how you can efficiently check if a required complement number has already been seen. Have you considered using a data structure that allows for quick lookups?

Think about how you can efficiently check if a complement to the current number exists in the array. Have you considered using a data structure that allows for quick lookups?

Tags

Good Problem Solving